Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Военный факультет
Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
НАСТОЛЬНОЕ ПРИЛОЖЕНИЕ ДЛЯ РАСЧЁТА ВОЗРАСТА ПОЛЬЗОВАТЕЛЯ В ПРИВЫЧНЫХ И НЕОЖИДАННЫХ ЕДИНИЦАХ ИЗМЕРЕНИЯ (ВАРИАНТ 20)

Студент: гр. 434701 Н.А.Шурхай
Руководитель: ассистент О.В. Внук

Минск 2026

СОДЕРЖАНИЕ

Ввведение 3
1 Проектый раздел 4
1.1 Архитектура приложения 4
1.2 Модель данных и структур 4
1.3 UML-диаграммы .....
1.3.1 Диаграммы последовательностей 5
1.3.3 Диаграммы активности 6
1.3.4 Диаграмма компонентов 7
1.4 Макеты интерфейсов 8
2 Рреализация проекта 10
2.1 Описание интерфейса и логики работы 10
2.2 Валидация введённых данных 10
3 Тестирование 11
3.1 План тестирования 11
3.2 Сценарии и акторы 11
3.3 Результаты тестирования 11
4 Системные требования 13
4.1 Функциональные требования 13
4.2 Нефункциональные требования 13
4.2.1 Атрибуты качества 13
Заключение 15
Список использованных источников 15
Приложение А (обязательное) листинг кода 16

ВВЕДЕНИЕ

Возраст человека принято измерять годами, месяцами и днями, однако такое представление не всегда наглядно передаёт масштаб прожитого времени. Более выразительным и увлекательным способом представления возраста является пересчёт количества прожитых дней в нестандартные, но интуитивно понятные единицы измерения – количество минут, чашек кофе, просмотренных серий сериалов или километров прокрученной ленты социальных сетей.

Целью данной работы является разработка настольного приложения «Возраст в необычных единицах», реализованного на языке C++ с непосредственным использованием функций Windows API (Win32), без применения сторонних GUI-фреймворков, и предназначенного для расчёта возраста пользователя по введённой дате рождения как в стандартных единицах (годы, месяцы, дни, минуты), так и в ряде нестандартных, шутливых единиц измерения.

В работе рассматриваются архитектура приложения, построенная по паттерну MVC в сочетании с паттерном «Наблюдатель» (Observer), модель данных и структур, UML-диаграммы, макеты интерфейса, а также вопросы реализации и тестирования разработанного программного обеспечения.

1 ПРОЕКТНЫЙ РАЗДЕЛ

1.1 Архитектура приложения

Приложение относится к классу настольных приложений с графическим интерфейсом пользователя и реализовано на языке C++ с непосредственным использованием функций Windows API (Win32). Архитектура программы построена по паттерну MVC (Model – View – Controller) в сочетании с паттерном «Наблюдатель» (Observer):

– Модель (класс AgeModel) хранит дату рождения, введённую пользователем, выполняет все вычисления и уведомляет подписанные представления об изменении данных через интерфейс IModelObserver;
– Представление (класс MainWindowView) отображает главное окно приложения с кнопкой запроса ввода данных и текстовым полем результата, подписывается на изменения модели через addObserver(this) и обновляет содержимое поля при получении уведомления onModelChanged();
– Контроллер (класс AgeController) обрабатывает запрос пользователя на ввод даты, отображает отдельное диалоговое окно ввода, выполняет валидацию введённой даты (проверка диапазона, високосного года, запрет будущей даты) и передаёт корректные данные модели.

Такое разделение обязанностей позволяет изменять правила валидации даты или добавлять новые «необычные» единицы измерения возраста без изменения кода отображения главного окна, а использование паттерна «Наблюдатель» допускает подключение дополнительных представлений (например, окна статистики) без изменения модели.

1.2 Модель данных и структур

Основной структурой данных, используемой для хранения результатов расчёта, является структура AgeStats, определённая в модуле AgeModel:

```
struct AgeStats {
    int years = 0;
    int months = 0;
    int days = 0;
    long long totalMinutes = 0;
    long long coffeeCups = 0;
    long long tvEpisodes = 0;
    double socialFeedKm = 0.0;
};
```

Класс AgeModel хранит единственный экземпляр AgeStats, а также последние введённые пользователем значения дня, месяца и года (lastDay, lastMonth, lastYear) – это позволяет при повторном открытии диалога предзаполнить поля ввода. Подписка представлений на модель реализована

через контейнер std::vector<IModelObserver*>, в который добавляются указатели на объекты, реализующие интерфейс IModelObserver с единственным виртуальным методом onModelChanged().

Перевод даты рождения и текущей даты в юлианский день (Julian Day Number) выполняется приватным методом AgeModel::toJdn(), что позволяет корректно вычислить количество прошедших дней totalDays независимо от различного количества дней в месяцах и високосности года. На основе totalDays рассчитываются все производные величины: totalMinutes = totalDays × 24 × 60, coffeeCups = totalDays × 2, tvEpisodes = totalDays × 1,5, socialFeedKm = totalDays × 0,08.

1.3 UML-диаграммы

1.3.1 Диаграммы последовательностей

Диаграмма последовательности для основного варианта использования отображает порядок вызовов между пользователем, главным окном (MainWindowView), контроллером (AgeController), диалоговым окном ввода даты (InputDlgProc) и моделью (AgeModel), включая альтернативную ветвь обработки некорректной даты.

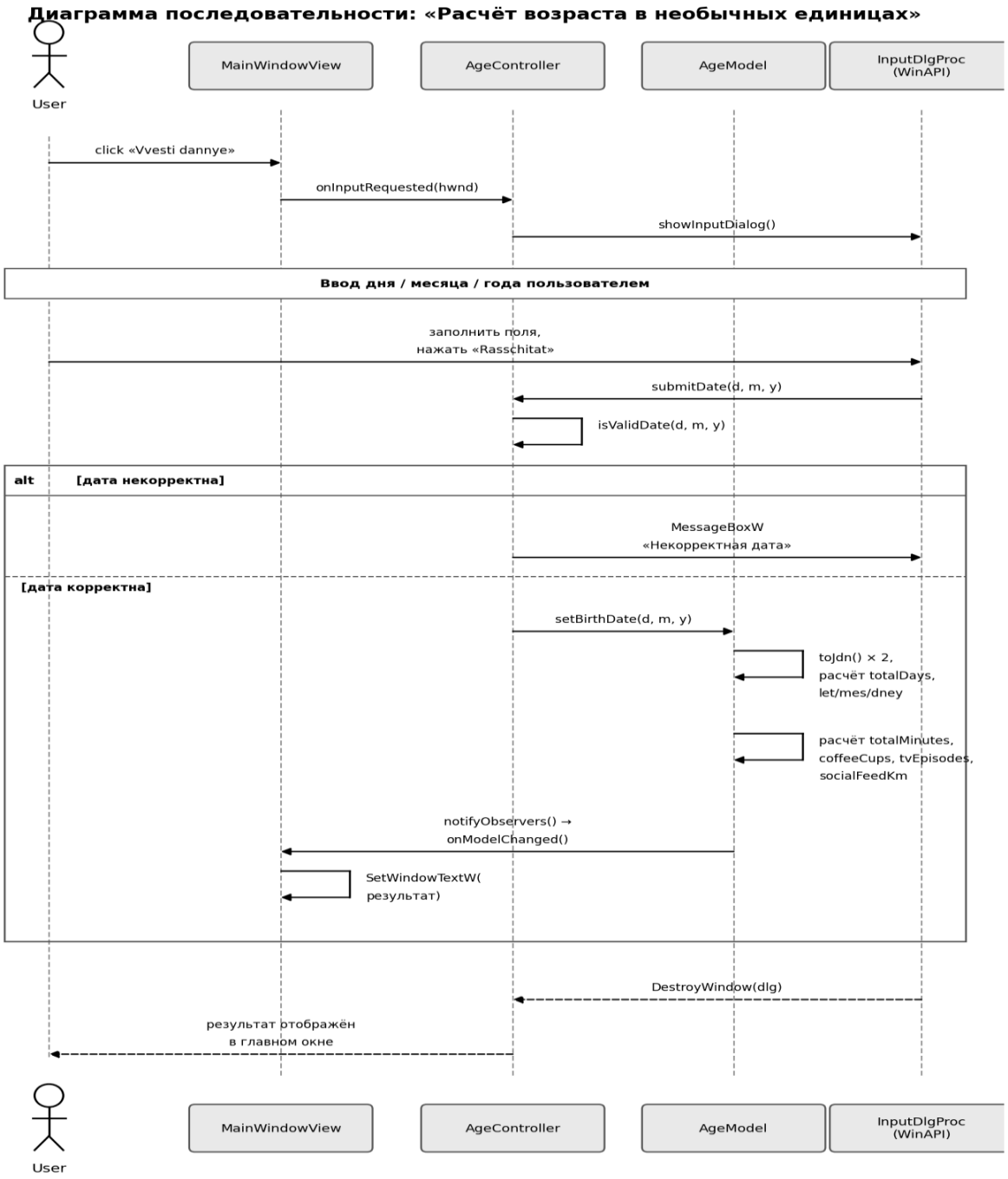


Рисунок 2 – Диаграмма последовательности для варианта использования «Расчёт возраста в необычных единицах»

1.3.3 Диаграммы активности

Диаграмма активности описывает пошаговую логику проверки введённой даты (диапазон года и месяца, корректность дня с учётом високосного года, запрет будущей даты) и последующего расчёта всех показателей возраста.

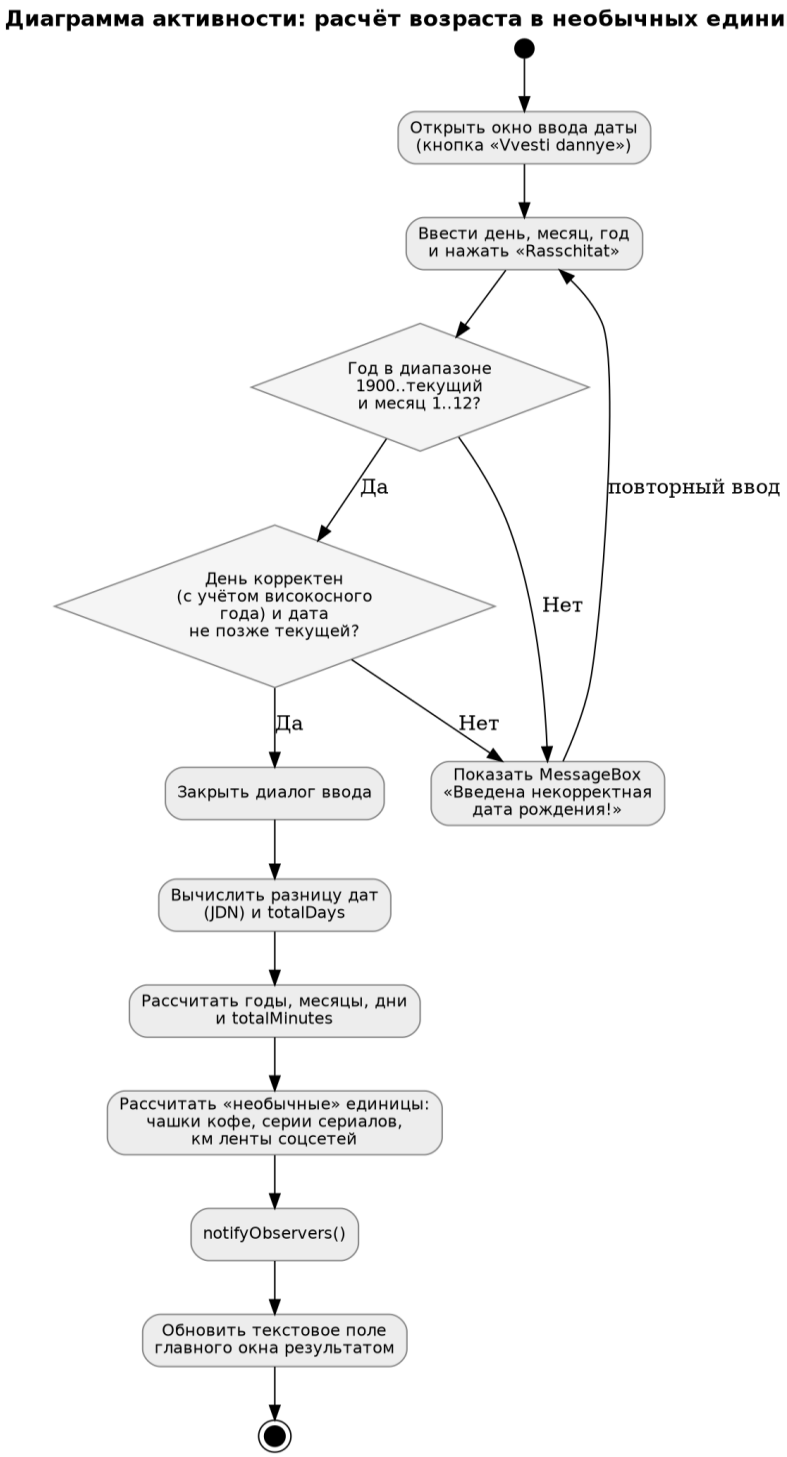


Рисунок 3 – Диаграмма активности расчёта возраста в необычных единицах

1.3.4 Диаграмма компонентов

Диаграмма компонентов иллюстрирует состав приложения (View, Controller, Model и интерфейс IModelObserver) и его взаимодействие с компонентами Windows API, используемыми для создания окон, получения текущей даты (GetLocalTime) и вывода сообщений (MessageBoxW). В отличие от клиент-серверных приложений, рассматриваемая программа не

взаимодействует с какими-либо внешними сетевыми сервисами — все вычисления выполняются локально, без обращения к сети.



Рисунок 4 – Диаграмма компонентов приложения «Возраст в необычных единицах»

1.4 Макеты интерфейсов

Для согласования внешнего вида и логики интерфейса были разработаны макеты главного окна приложения и диалогового окна ввода даты рождения.

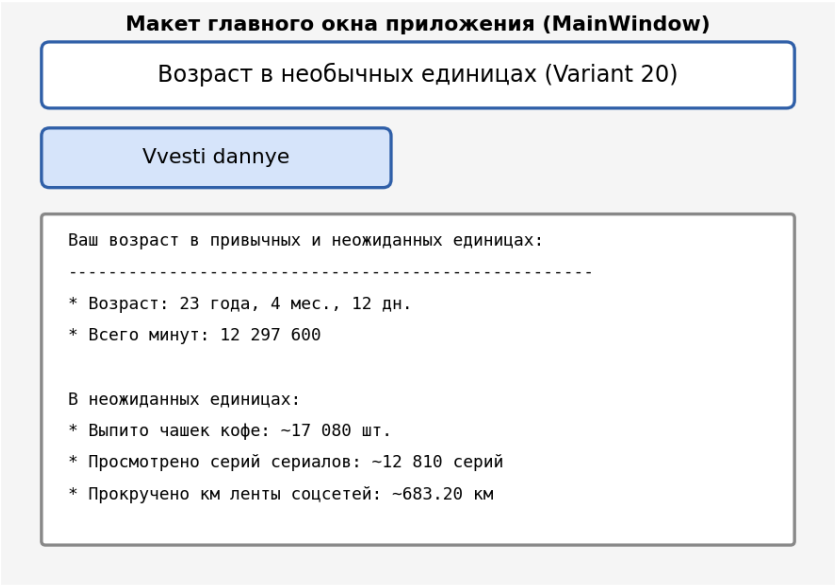


Рисунок 5 – Макет главного окна приложения (MainWindow)

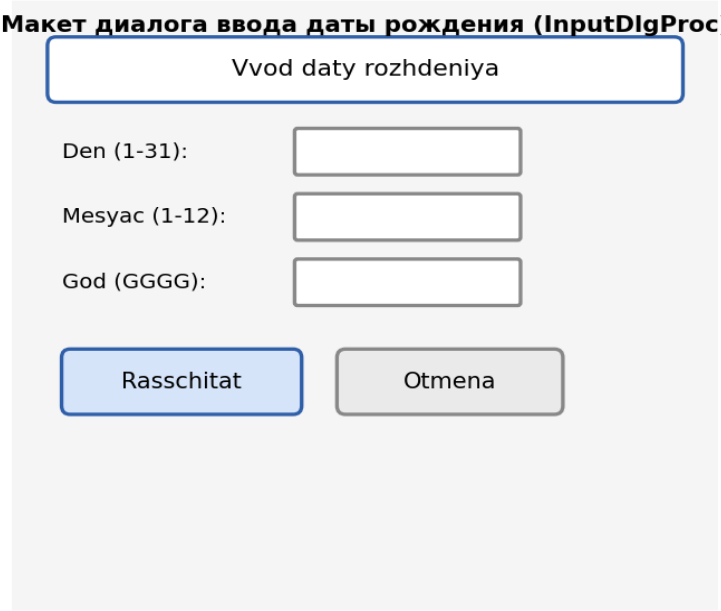


Рисунок 6 – Макет диалогового окна ввода даты рождения (InputDlgProc)

2 РЕАЛИЗАЦИЯ ПРОЕКТА

2.1 Описание интерфейса и логики работы

Главное окно приложения MainWindow содержит кнопку «Vvesti dannye» и многострочное текстовое поле только для чтения (ES_MULTILINE | ES_READONLY), в котором отображается результат расчёта. При нажатии кнопки контроллер создаёт и отображает отдельное окно диалога ввода даты рождения, реализованное как самостоятельный класс окна Win32 (InputDateDialogClass) с собственным циклом обработки сообщений, при этом главное окно временно блокируется вызовом EnableWindow(parent, FALSE) и восстанавливается после закрытия диалога.

Диалоговое окно содержит три однострочных поля ввода (день, месяц, год) с ограничением стиля ES_NUMBER, предзаполненные последними введёнными пользователем значениями через ctx->model->getLastInput(), а также кнопки «Rasschitat» (IDOK) и «Otmena» (IDCANCEL). При нажатии кнопки «Rasschitat» выполняется преобразование введённого текста в числа функцией _wtoi() и вызывается метод контроллера submitDate(), который проверяет корректность полученной даты.

2.2 Валидация введённых данных

Проверка корректности даты выполняется методом AgeController::isValidDate() и включает:

– проверку диапазона года (от 1900 до текущего года, определяемого функцией GetLocalTime);
– проверку диапазона месяца (от 1 до 12);
– проверку диапазона дня с учётом фактического количества дней в соответствующем месяце и високосности года, определяемой по формуле (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
– проверку того, что введённая дата не превышает текущую дату (запрет указания даты рождения «в будущем»).

При обнаружении некорректных данных выводится модальное сообщение MessageBoxW с текстом «Введена некорректная дата рождения! Проверьте значения дня, месяца и года.», диалоговое окно не закрывается, а ранее введённые пользователем значения не сбрасываются.

3 ТЕСТИРОВАНИЕ

3.1 План тестирования

Целью тестирования являлась проверка корректности работы приложения в трёх аспектах: навигация между главным окном и диалогом ввода даты, правильность математических расчётов возраста и производных «необычных» единиц измерения, а также устойчивость программы к некорректному вводу данных (пустые поля, нечисловые значения, несуществующие календарные даты, дата рождения в будущем). Тестирование проводилось методом чёрного ящика на этапе, когда приложение уже успешно собиралось и все окна открывались без ошибок. Тестирование считалось завершённым после того, как все основные сценарии были пройдены без критических ошибок, а выявленные замечания — устранены.

3.2 Сценарии и акторы

Единственным актором системы является пользователь, взаимодействующий с графическим интерфейсом приложения — в отличие от клиент-серверных систем, внешние акторы (серверы, внешние API) в данном приложении отсутствуют, все вычисления выполняются локально. Проверялись следующие сценарии: запуск приложения, открытие диалога ввода даты, ввод корректной даты рождения и получение результата, повторное открытие диалога с предзаполненными полями, ввод некорректных данных (несуществующий день месяца, 29 февраля не в високосном году, дата в будущем, пустые поля) и корректная реакция программы на такие данные, а также закрытие диалога кнопкой «Otmena» и закрытие главного окна приложения.

3.3 Результаты тестирования

Итоговая сводка выполненных тестов приведена в таблице 1.

Таблица 1 – Результаты тестирования приложения «Возраст в необычных единицах»

№	Сценарий	Ожидаемый результат	Результат
1	Запуск приложения, отображение главного окна	Окно открывается, поле результата содержит текст-подсказку	Пройден

№	Сценарий	Ожидаемый результат	Результат
2	Открытие диалога ввода по кнопке «Vvesti dannye»	Диалог открывается, главное окно блокируется (EnableWindow FALSE)	Пройден
3	Ввод корректной даты рождения (например, 15.06.2002)	Диалог закрывается; в главном окне отображается возраст и все производные величины	Пройден
4	Ввод дня 31 для месяца с 30 днями (например, 31.04.2000)	Выводится MessageBox с сообщением об ошибке, диалог остаётся открытым	Пройден
5	Ввод даты 29.02 для невисокосного года	Выводится MessageBox с сообщением об ошибке	Пройден
6	Ввод даты позже текущей (будущая дата рождения)	Выводится MessageBox с сообщением об ошибке	Пройден
7	Повторное открытие диалога после успешного расчёта	Поля дня/месяца/года предзаполнены последними введёнными значениями	Пройден
8	Пустые поля ввода при нажатии «Rasschitat»	_wtoi() возвращает 0; дата 0.0.0 распознаётся как некорректная, выводится сообщение об ошибке	Пройден
9	Нажатие кнопки «Otmena» в диалоге ввода	Диалог закрывается без изменения данных модели, главное окно разблокируется	Пройден
10	Закрытие главного окна (крестик / Alt+F4)	Приложение штатно завершает работу (PostQuitMessage)	Пройден

В ходе тестирования критических дефектов, приводящих к аварийному завершению программы или зависанию интерфейса, не выявлено. Обнаруженное замечание – отображение надписей интерфейса в транслитерированном виде (латиницей) вместо кириллицы вследствие особенностей кодировки строковых литералов исходного файла – отнесено к косметическим недостаткам, не влияющим на работоспособность приложения, и рекомендовано к устранению в дальнейших версиях программы. На основании полученных результатов можно сделать вывод, что приложение стабильно работает во всех основных сценариях и корректно обрабатывает как правильные, так и ошибочные пользовательские данные.

4 СИСТЕМНЫЕ ТРЕБОВАНИЯ

FR-001: Система должна после запуска отображать главное окно с кнопкой «Vvesti dannye» и текстовым полем результата, содержащим подсказку до выполнения первого расчёта.

FR-002: Система должна открывать диалоговое окно ввода даты рождения при нажатии кнопки «Vvesti dannye» и блокировать главное окно (EnableWindow) до закрытия диалога.

FR-003: Диалоговое окно должно предоставлять три числовых поля ввода – день, месяц, год, предзаполненные последними введёнными пользователем значениями.

FR-004: Система должна проверять корректность введённой даты (диапазон года 1900 – текущий, диапазон месяца 1–12, корректность дня с учётом високосного года, запрет будущей даты) при нажатии кнопки «Rasschitat».

FR-005: При некорректных данных система должна выводить модальное сообщение с понятным текстом ошибки и не выполнять расчёт, сохраняя диалог открытым.

FR-006: При корректных данных система должна вычислять возраст пользователя в годах, месяцах и днях, а также общее количество минут, прожитых с даты рождения.

FR-007: Система должна вычислять и отображать производные «необычные» единицы измерения: количество чашек кофе (из расчёта 2 в сутки), количество просмотренных серий сериалов (из расчёта 1,5 в сутки) и количество километров прокрученной ленты социальных сетей (из расчёта 0,08 км в сутки).

FR-008: Система должна отображать результат расчёта в текстовом поле главного окна в структурированном, читаемом виде с разделением на «привычные» и «неожиданные» единицы измерения.

FR-009: Система должна закрывать диалоговое окно ввода и разблокировать главное окно при нажатии кнопки «Otmena» либо после успешного расчёта.

FR-010: Система должна завершать работу приложения при закрытии главного окна (нажатием на системную кнопку закрытия).

4.2 Нефункциональные требования

4.2.1 Атрибуты качества

Надёжность: система должна устойчиво работать при типичных пользовательских ошибках (пустые поля, нечисловой ввод, несуществующие даты), не допуская аварийного завершения приложения. Важность: высокая. Измерение: отсутствие падений приложения при выполнении основных сценариев и количество необработанных ошибок ввода.

Производительность: расчёт возраста и всех производных величин выполняется локально без обращения к сети и должен завершаться менее чем

за 1 секунду на целевых устройствах; открытие диалогового окна не должно вызывать заметных задержек интерфейса. Важность: средняя. Измерение: время отклика на нажатие кнопок «Vvesti dannye» и «Rasschitat».

Удобство использования: интерфейс должен быть простым и понятным без необходимости изучения документации. Отдельным замечанием является то, что надписи интерфейса представлены в транслитерированном виде (латиницей) вследствие особенностей кодировки исходного файла, что снижает удобство восприятия для русскоязычного пользователя и рекомендуется к исправлению. Важность: высокая. Измерение: количество шагов до получения результата и субъективная оценка удобства при демонстрации проекта.

Совместимость: приложение использует только базовые функции Windows API (User32, Kernel32) без сторонних зависимостей и должно корректно работать на операционных системах Windows 10 и Windows 11 (x86/x64). Важность: высокая. Измерение: успешный запуск и работа программы на нескольких тестовых конфигурациях Windows.

Поддерживаемость и расширяемость: архитектура приложения построена по паттерну MVC с использованием паттерна «Наблюдатель», что позволяет добавлять новые «необычные» единицы измерения возраста или дополнительные представления без существенной переработки модели и контроллера. Важность: средняя для конечного пользователя, высокая для разработчика. Измерение: время, необходимое для добавления новой единицы измерения без нарушения работы существующего функционала.

Безопасность: приложение не обрабатывает конфиденциальные данные пользователя, не выполняет сетевых операций и не сохраняет данные о введённой дате рождения на диске после завершения работы. Важность: низкая-средняя, достаточная для текущего проекта. Измерение: отсутствие передачи и сохранения персональных данных пользователя.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы было разработано настольное приложение «Возраст в необычных единицах», предназначенное для расчёта возраста пользователя по введённой дате рождения как в стандартных единицах (годы, месяцы, дни, минуты), так и в ряде нестандартных, шутливых единиц измерения (чашки кофе, серии сериалов, километры ленты социальных сетей). Приложение реализовано на языке C++ с использованием функций Windows API и построено по архитектуре MVC с применением паттерна «Наблюдатель».

В проектной части были разработаны UML-диаграммы, описывающие архитектуру приложения, поведение окон и последовательность взаимодействий с пользователем, а также макеты интерфейса главного окна и диалога ввода даты. На этапе реализации выполнено создание графического интерфейса средствами Win32 API и программной логики расчёта возраста и производных величин. Проведено функциональное тестирование методом чёрного ящика, подтвердившее работоспособность и корректность основных сценариев использования, включая обработку некорректного ввода.

Разработанное приложение может быть использовано в учебных целях для иллюстрации жизненного цикла разработки программного обеспечения, применения архитектурных паттернов MVC и «Наблюдатель», а также в качестве развлекательного инструмента для наглядного представления возраста пользователя в необычных единицах измерения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Документация Microsoft: Windows API (Win32 API). Официальный портал Microsoft Learn.
2. Шилдт Г. C++: руководство для начинающих. – М.: Вильямс, 2019.
3. Фримен Э., Робсон Э. Паттерны проектирования (Head First. Design Patterns). – СПб.: Питер, 2021.
4. Рихтер Д. Windows via C/C++. Программирование на языке C++ с использованием функций Windows API. – СПб.: Питер, 2020.

Приложение А
(обязательное)
Листинг кода

```
#include <windows.h>
#include <string>
#include <vector>
#include <sstream>
#include <iomanip>

// ============================================
// 1. PATTERN OBSERVER (DLYA AKTIVNOY MODELI)
// ============================================
class IModelObserver {
public:
    virtual ~IModelObserver() = default;
    virtual void onModelChanged() = 0;
};

// ============================================
// 2. MODEL
// ============================================
struct AgeStats {
    int years = 0;
    int months = 0;
    int days = 0;
    long long totalMinutes = 0;
    long long coffeeCups = 0;
    long long tvEpisodes = 0;
    double socialFeedKm = 0.0;
};

class AgeModel {
private:
    int lastDay = 1;
    int lastMonth = 1;
    int lastYear = 2000;
    bool hasData = false;
    AgeStats stats;
    std::vector<IModelObserver*> observers;

    long long toJdn(int y, int m, int d) const {
        int a = (14 - m) / 12;
        long long y_adj = y + 4800 - a;
        int m_adj = m + 12 * a - 3;
```

```
        return d + (153 * m_adj + 2) / 5 + 365 * y_adj +
y_adj / 4 - y_adj / 100 + y_adj / 400 - 32045;
    }

    void notifyObservers() {
        for (auto* obs : observers) {
            obs->onModelChanged();
        }
    }

public:
    void addObserver(IModelObserver* obs) {
        observers.push_back(obs);
    }

    void setBirthDate(int day, int month, int year) {
        lastDay = day;
        lastMonth = month;
        lastYear = year;

        SYSTEMTIME st;
        GetLocalTime(&st);

        long long birthJdn = toJdn(year, month, day);
        long   long   currentJdn   =   toJdn(st.wYear,
st.wMonth, st.wDay);
        long long totalDays = currentJdn - birthJdn;

        int y = st.wYear - year;
        int m = st.wMonth - month;
        int d = st.wDay - day;

        if (d < 0) {
            m -= 1;
            d += 30;
        }
        if (m < 0) {
            y -= 1;
            m += 12;
        }

        stats.years = y;
        stats.months = m;
        stats.days = d;
        stats.totalMinutes = totalDays * 24 * 60;
```

```
        stats.coffeeCups = totalDays * 2;
        stats.tvEpisodes       =       static_cast<long
long>(totalDays * 1.5);
        stats.socialFeedKm = totalDays * 0.08;

        hasData = true;
        notifyObservers();
    }

    bool getHasData() const { return hasData; }
    AgeStats getStats() const { return stats; }
    void getLastInput(int& d, int& m, int& y) const {
        d = lastDay;
        m = lastMonth;
        y = lastYear;
    }
};

class AgeController;

// ============================================
// 3. VIEW
// ============================================
class MainWindowView : public IModelObserver {
private:
    HWND hMainWnd = nullptr;
    HWND hBtnInput = nullptr;
    HWND hTextResult = nullptr;
    AgeModel& model;
    AgeController* controller = nullptr;

public:
    MainWindowView(AgeModel& m) : model(m) {
        model.addObserver(this);
    }

    void setController(AgeController* c) { controller =
c; }
    void setHwnd(HWND hwnd) { hMainWnd = hwnd; }
    HWND getHwnd() const { return hMainWnd; }

    void createControls(HWND hwnd) {
        hMainWnd = hwnd;

        hBtnInput = CreateWindowW(
```

```
            L"BUTTON", L"Vvesti dannye",
            WS_TABSTOP  |  WS_VISIBLE  |  WS_CHILD  |
BS_DEFPUSHBUTTON,
            30, 20, 160, 35,
            hwnd,                                (HMENU)101,
(HINSTANCE)GetWindowLongPtr(hwnd,      GWLP_HINSTANCE),
nullptr
        );

        hTextResult = CreateWindowW(
            L"EDIT", L"Nazhmite 'Vvesti dannye', chtoby
rasschitat vozrast.",
            WS_CHILD  |  WS_VISIBLE  |  WS_BORDER  |
ES_MULTILINE | ES_READONLY | ES_AUTOVSCROLL,
            30, 70, 480, 240,
            hwnd,                                (HMENU)102,
(HINSTANCE)GetWindowLongPtr(hwnd,      GWLP_HINSTANCE),
nullptr
        );

        HFONT                    hFont                =
(HFONT)GetStockObject(DEFAULT_GUI_FONT);
        SendMessageW(hBtnInput,              WM_SETFONT,
(WPARAM)hFont, TRUE);
        SendMessageW(hTextResult,            WM_SETFONT,
(WPARAM)hFont, TRUE);
    }

    void onModelChanged() override {
        if (!model.getHasData()) return;

        AgeStats s = model.getStats();
        std::wstringstream ss;
        ss << L"Vash vozrast v privychnyh i neozhidannyh
edinicah:\r\n"
           <<  L"-------------------------------------
-------------\r\n"
           << L"* Vozrast: " << s.years << L" let, " <<
s.months << L" mes., " << s.days << L" dn.\r\n"
           << L"* Vsego minut: " << s.totalMinutes <<
L"\r\n\r\n"
           << L"V neozhidannyh edinicah:\r\n"
           <<  L"*  Vypito  chashek  kofe:  ~"  <<
s.coffeeCups << L" sht.\r\n"
```

```
            << L"* Prosmotreno seriy serialov: ~" <<
s.tvEpisodes << L" seriy\r\n"
            << L"* Prokrucheno kilometrov lenty
socsetey: ~"
            << std::fixed << std::setprecision(2) <<
s.socialFeedKm << L" km\r\n";

        SetWindowTextW(hTextResult, ss.str().c_str());
    }
};

// ============================================
// 4. CONTROLLER
// ============================================
class AgeController {
private:
    AgeModel& model;
    MainWindowView& view;

    bool isLeapYear(int y) const {
        return (y % 4 == 0 && y % 100 != 0) || (y % 400
== 0);
    }

    bool isValidDate(int d, int m, int y) const {
        SYSTEMTIME st;
        GetLocalTime(&st);

        if (y < 1900 || y > st.wYear) return false;
        if (m < 1 || m > 12) return false;

        int daysInMonth[] = { 31, 28, 31, 30, 31, 30, 31,
31, 30, 31, 30, 31 };
        if (m == 2 && isLeapYear(y)) daysInMonth[1] = 29;

        if (d < 1 || d > daysInMonth[m - 1]) return false;

        if (y == st.wYear) {
            if (m > st.wMonth) return false;
            if (m == st.wMonth && d > st.wDay) return
false;
        }
        return true;
    }
```

```
public:
    AgeController(AgeModel& m, MainWindowView& v) :
model(m), view(v) {}

    void onInputRequested(HWND parent) {
        showInputDialog(parent);
    }

    bool submitDate(int d, int m, int y, HWND dlgHwnd) {
        if (!isValidDate(d, m, y)) {
            MessageBoxW(dlgHwnd,
                L"Vvedena nekorrektnaya data
rozhdeniya!\nProverte znacheniya dnya, mesyaca i goda.",
                L"Oshibka vvoda", MB_OK | MB_ICONERROR);
            return false;
        }
        model.setBirthDate(d, m, y);
        return true;
    }

    void showInputDialog(HWND parent);
};

// ========================================
// 5. DIALOG VVODA
// ========================================
struct DialogContext {
    AgeController* controller;
    AgeModel* model;
};

LRESULT CALLBACK InputDlgProc(HWND hwnd, UINT msg, WPARAM
wParam, LPARAM lParam) {
    static DialogContext* ctx = nullptr;
    static HWND hEditDay, hEditMonth, hEditYear;

    switch (msg) {
    case WM_CREATE: {
        CREATESTRUCT* cs = (CREATESTRUCT*)lParam;
        ctx = (DialogContext*)cs->lpCreateParams;

        HFONT hFont =
(HFONT)GetStockObject(DEFAULT_GUI_FONT);
```

```
        CreateWindowW(L"STATIC",     L"Den     (1-31):",
WS_CHILD | WS_VISIBLE, 20, 20, 100, 20, hwnd, nullptr,
nullptr, nullptr);
        hEditDay = CreateWindowW(L"EDIT", L"", WS_CHILD
| WS_VISIBLE | WS_BORDER | ES_NUMBER, 130, 20, 80, 22,
hwnd, (HMENU)201, nullptr, nullptr);

        CreateWindowW(L"STATIC",   L"Mesyac   (1-12):",
WS_CHILD | WS_VISIBLE, 20, 55, 100, 20, hwnd, nullptr,
nullptr, nullptr);
        hEditMonth    =    CreateWindowW(L"EDIT",    L"",
WS_CHILD | WS_VISIBLE | WS_BORDER | ES_NUMBER, 130, 55,
80, 22, hwnd, (HMENU)202, nullptr, nullptr);

        CreateWindowW(L"STATIC",     L"God     (GGGG):",
WS_CHILD | WS_VISIBLE, 20, 90, 100, 20, hwnd, nullptr,
nullptr, nullptr);
        hEditYear = CreateWindowW(L"EDIT", L"", WS_CHILD
| WS_VISIBLE | WS_BORDER | ES_NUMBER, 130, 90, 80, 22,
hwnd, (HMENU)203, nullptr, nullptr);

        HWND    hBtnOk    =    CreateWindowW(L"BUTTON",
L"Rasschitat", WS_CHILD | WS_VISIBLE | BS_DEFPUSHBUTTON,
30, 135, 90, 30, hwnd, (HMENU)IDOK, nullptr, nullptr);
        HWND   hBtnCancel   =   CreateWindowW(L"BUTTON",
L"Otmena", WS_CHILD | WS_VISIBLE, 130, 135, 80, 30, hwnd,
(HMENU)IDCANCEL, nullptr, nullptr);

        SendMessageW(hEditDay,                WM_SETFONT,
(WPARAM)hFont, TRUE);
        SendMessageW(hEditMonth,              WM_SETFONT,
(WPARAM)hFont, TRUE);
        SendMessageW(hEditYear,               WM_SETFONT,
(WPARAM)hFont, TRUE);
        SendMessageW(hBtnOk, WM_SETFONT, (WPARAM)hFont,
TRUE);
        SendMessageW(hBtnCancel,              WM_SETFONT,
(WPARAM)hFont, TRUE);

        int ld, lm, ly;
        ctx->model->getLastInput(ld, lm, ly);
        SetWindowTextW(hEditDay,
std::to_wstring(ld).c_str());
        SetWindowTextW(hEditMonth,
std::to_wstring(lm).c_str());
```

```
        SetWindowTextW(hEditYear,
std::to_wstring(ly).c_str());
        return 0;
    }
    case WM_COMMAND: {
        int id = LOWORD(wParam);
        if (id == IDOK) {
            wchar_t bDay[16], bMonth[16], bYear[16];
            GetWindowTextW(hEditDay, bDay, 16);
            GetWindowTextW(hEditMonth, bMonth, 16);
            GetWindowTextW(hEditYear, bYear, 16);

            int d = _wtoi(bDay);
            int m = _wtoi(bMonth);
            int y = _wtoi(bYear);

            if  (ctx->controller->submitDate(d,  m,  y,
hwnd)) {
                DestroyWindow(hwnd);
            }
        }
        else if (id == IDCANCEL) {
            DestroyWindow(hwnd);
        }
        return 0;
    }
    case WM_CLOSE:
        DestroyWindow(hwnd);
        return 0;
    default:
        return   DefWindowProcW(hwnd,   msg,   wParam,
lParam);
    }
}

void AgeController::showInputDialog(HWND parent) {
    DialogContext ctx = { this, &model };

    WNDCLASSW wc = { 0 };
    wc.lpfnWndProc = InputDlgProc;
    wc.hInstance = GetModuleHandle(nullptr);
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.lpszClassName = L"InputDateDialogClass";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);
    RegisterClassW(&wc);
```

```
    HWND hDlg = CreateWindowExW(
        WS_EX_DLGMODALFRAME,
        L"InputDateDialogClass",       L"Vvod       daty
rozhdeniya",
        WS_POPUP | WS_CAPTION | WS_SYSMENU | WS_VISIBLE,
        CW_USEDEFAULT, CW_USEDEFAULT, 260, 220,
        parent, nullptr, wc.hInstance, &ctx
    );

    EnableWindow(parent, FALSE);
    MSG msg;
    while (IsWindow(hDlg) && GetMessageW(&msg, nullptr,
0, 0)) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }
    EnableWindow(parent, TRUE);
    SetForegroundWindow(parent);
}

// ========================================
// 6. WINMAIN & GLAVNYY CIKL
// ========================================
static AgeController* g_controller = nullptr;
static MainWindowView* g_view = nullptr;

LRESULT CALLBACK MainWndProc(HWND hwnd, UINT msg, WPARAM
wParam, LPARAM lParam) {
    switch (msg) {
    case WM_CREATE:
        if (g_view) g_view->createControls(hwnd);
        return 0;
    case WM_COMMAND:
        if (LOWORD(wParam) == 101 && g_controller) {
            g_controller->onInputRequested(hwnd);
        }
        return 0;
    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    default:
        return   DefWindowProcW(hwnd,   msg,   wParam,
lParam);
    }
```

```
}

int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE,
PWSTR, int nCmdShow) {
    AgeModel model;
    MainWindowView view(model);
    AgeController controller(model, view);

    view.setController(&controller);
    g_controller = &controller;
    g_view = &view;

    const wchar_t CLASS_NAME[] =
L"AgeCalculatorMainWindow";

    WNDCLASSW wc = { 0 };
    wc.lpfnWndProc = MainWndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW);
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    RegisterClassW(&wc);

    HWND hwnd = CreateWindowExW(
        0, CLASS_NAME, L"Vozrast v neozhidannyh edinicah
(Variant 20)",
        WS_OVERLAPPED | WS_CAPTION | WS_SYSMENU |
WS_MINIMIZEBOX,
        CW_USEDEFAULT, CW_USEDEFAULT, 560, 370,
        nullptr, nullptr, hInstance, nullptr
    );

    if (!hwnd) return 0;

    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);

    MSG msg = {};
    while (GetMessageW(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    return 0;
```

```
}
```